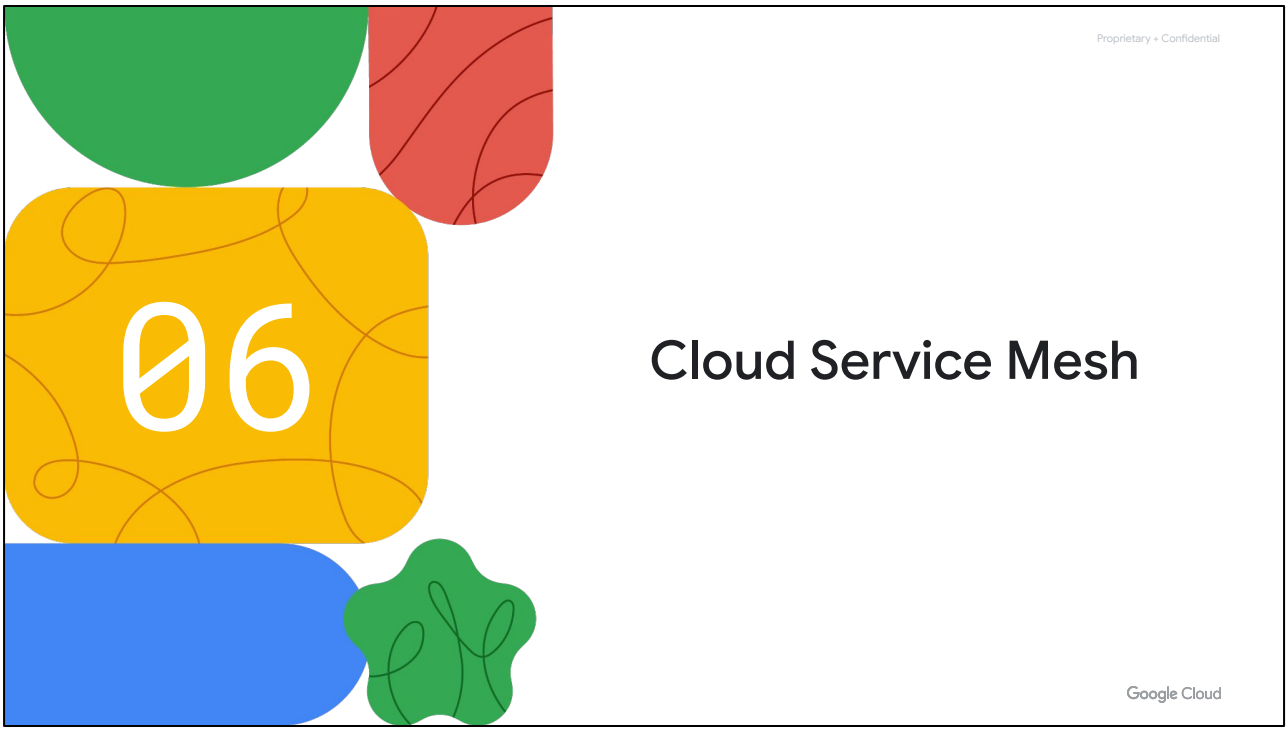


Traditionally, developers had to embed networking code directly into their applications. This can become complex quickly, especially when working with microservices.

To address this challenge, Cloud Service Mesh (CSM) abstracts away the complexities of network management.

Proprietary + Confidential

Cloud Service Mesh

01 Introduction to Cloud Service Mesh

02 Provisioning Cloud Service Mesh

03 Handling requests

04 Cloud Service Mesh dashboards and support

Google Cloud

In this section, titled “Cloud Service Mesh,” you learn to:

- List and describe the benefits of using Cloud Service Mesh.
- Install and configure Cloud Service Mesh on different clusters
- Trace the path of a request through the mesh, correctly identifying and explaining the role of key components like Envoy proxies, Mesh CA, and extensions in handling the request.
- Create Service Mesh Dashboards from workload telemetry including metrics, traces, and logs.

Proprietary + Confidential

Cloud Service Mesh

01 Introduction to Cloud Service Mesh

02 Provisioning Cloud Service Mesh

03 Handling requests

04 Cloud Service Mesh dashboards and support

Google Cloud

Let’s begin with an introduction to Cloud Service Mesh.

Proprietary + Confidential

Service-to-service interaction with a service mesh

A **Service Mesh** is a dedicated infrastructure layer designed to handle service-to-service communication in applications running on Kubernetes.

Google Cloud

A **service mesh** is a transparent and intelligent network that sits alongside your application code, managing how your various services interact with each other. It's not the application itself, but rather the underlying logic that makes distributed systems reliable, secure, and observable.

Proprietary + Confidential

A service mesh abstracts away cross-cutting challenges

- Consistent and centralized traffic management and security policy enforcement.
- Automatic collection of metrics, logs, and traces.
- Sophisticated traffic management rules without application code changes.
- Centralized configuration and enforcement of resilience patterns.
- Consistent set of policies and behaviors across the entire application.

Google Cloud

A Service Mesh addresses these complexities by abstracting away these cross-cutting concerns from the individual services.

- Consistent and centralized traffic management
- Centralized and consistent enforcement of security policies
- Automatic collection of metrics, logs, and traces.
- Sophisticated traffic management rules without application code changes.
- Centralized configuration and enforcement of resilience patterns.
- Enforces a consistent set of policies and behaviors across the entire application.

Proprietary + Confidential

Service Mesh is made of two layers

Data plane

Composed of lightweight proxy instances that are deployed as sidecars.

Control plane

Acts as the brain of the service mesh and manages and configures the data plane proxies.

Google Cloud

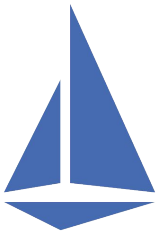
A typical Service Mesh architecture is divided into two distinct layers:

- **Data Plane:** This layer is composed of lightweight proxy instances (often based on Envoy) that are deployed as sidecars alongside each application service instance. The data plane is responsible for the "how" of service communication.

- **Control Plane:** This layer acts as the brain of the service mesh. It manages and configures the data plane proxies. It provides APIs for defining traffic routing rules, security policies, and observability configurations. It's responsible for the "what" and "why" of service communication.

Proprietary + Confidential

Istio is an open-source service mesh



Istio

Istio features include:
- Traffic Management
- Security
- Observability
- Policy Enforcement

Istio's architecture relies on Envoy as its data plane proxy.

Google Cloud

Istio is a popular and powerful open-source service mesh implementation for Kubernetes . It provides a comprehensive set of features, including:

- Traffic Management
- Security
- Observability
- Policy Enforcement

Istio's architecture heavily relies on Envoy as its data plane proxy and provides a feature-rich control plane to manage and configure these proxies.

Proprietary + Confidential

Cloud Service Mesh is managed by Google

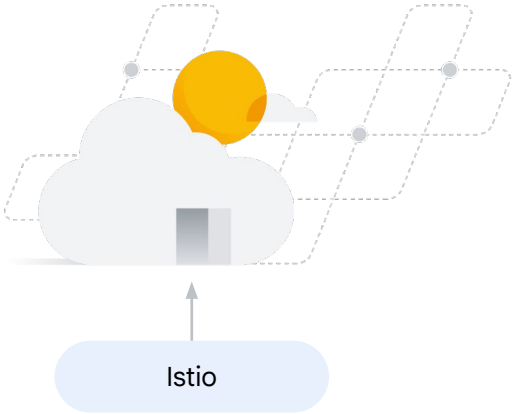
Google handles upgrades, scaling, and security across the control plane, data plane, and related resources.

Google Cloud

Cloud Service Mesh, or CSM, is managed by Google, meaning Google handles upgrades, scaling, and security, minimizing manual maintenance. This covers the control plane, data plane, and related resources.

Proprietary + Confidential

CSM is built on top of Istio



The open-source Istio APIs provide a way to interact with and configure the service mesh.

If you use open source Istio APIs, your data plane is provided by Envoy proxies.

Google Cloud

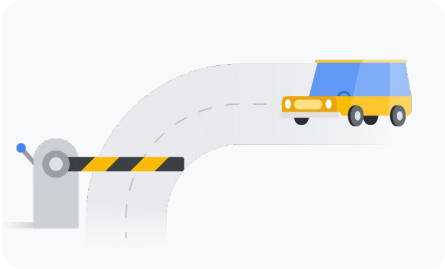
CSM is built on Istio, an open-source service mesh technology.

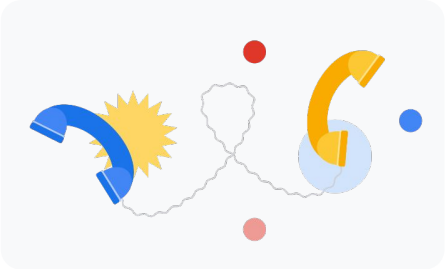
The open-source Istio APIs provide a way to interact with and configure the service mesh.

If you use open source Istio APIs, your data plane is provided by Envoy proxies.

Proprietary + Confidential

Envoy proxies and proxyless gRPC applications





Envoy proxies

Act as intermediaries, handling network traffic to and from your application.

Proxyless gRPC applications

Directly handle communication using gRPC, a high-performance framework often used in microservices for efficient service-to-service communication.

Google Cloud

When using Google Cloud APIs, your data plane can be managed in two ways:

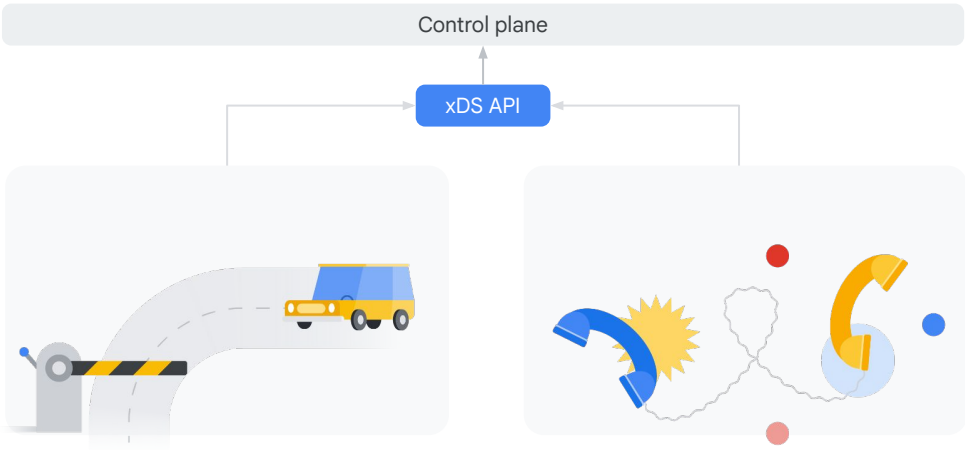
- **Envoy proxies** act as intermediaries, handling network traffic to and from your application.
- **Proxyless gRPC applications** directly handle communication using gRPC, a high-performance framework often used in microservices for efficient service-to-service communication.

Proxyless gRPC applications help avoid the overhead of running a sidecar.

-

Proprietary + Confidential

The xDS API is used to connect to the control plane

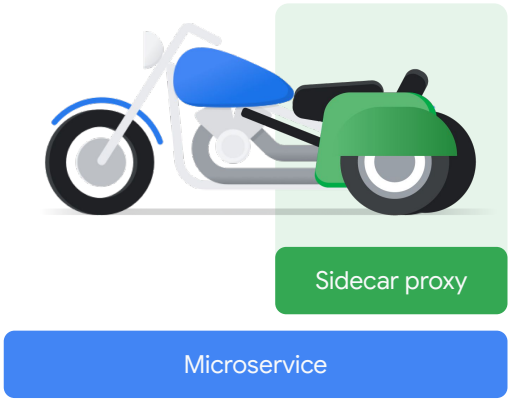


Google Cloud

Envoy proxies and proxyless gRPC both use the xDS API to connect to the control plane. If you use proxyless gRPC, you have a choice of supported languages for your applications, including Go, C++, Java, and Python.

Proprietary + Confidential

Sidecar proxies



These proxies, managed by the mesh, handle tasks like:

- Service discovery
- Traffic routing and splitting
- Security policy enforcement
- Monitoring and tracing

Google Cloud

CSM deploys a lightweight sidecar proxy along each microservice, which acts like a smart intermediary for all network communication.

These proxies, managed by the mesh, handle tasks like:

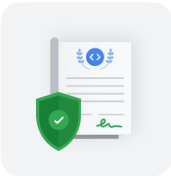
- Service discovery
- Traffic routing and splitting
- Security policy enforcement
- Monitoring and tracing

Proprietary + Confidential

Sidecar proxy advantages

Cleaner code

Agility



Consistency

Security

Traffic management

Observability

Google Cloud

This approach has several advantages:

Proprietary + Confidential

Cleaner code

- Developers can focus on business logic instead of networking complexities.
- Reduced development time.
- Improvements can be made to readability and maintainability of the code.

Google Cloud

One advantage is that it allows developers to focus on business logic instead of networking complexities. Development time is reduced and improvements can be made to readability and maintainability of the code.

Proprietary + Confidential

Increased agility

- Proxies increase agility.
- You can update network behavior without needing to redeploy applications.

Google Cloud

Proxies also increase agility. You can update network behavior without needing to redeploy applications. For example if you must change retry logic or adjust security rules, you can simply configure the mesh to accommodate these updates.

Proprietary + Confidential

Consistency

- Apply policies consistently across services.
- Ensure security and reliability.

Google Cloud

Proxies also enable you to apply policies consistently across services, ensuring security and reliability.

Proprietary + Confidential

Enhanced security

- CSM supports a zero-trust model.
- CSM automatically encrypts traffic between services using mutual TLS.
- Central security rule management.

Google Cloud

On the topic of security, CSM supports a zero-trust model, meaning every service-to-service interaction is authenticated and authorized.

CSM automatically encrypts traffic between services using mutual TLS, protecting sensitive data in transit.

And you can manage security rules centrally instead of configuring each service individually.

Proprietary + Confidential

Advanced traffic management

- Traffic Routing can be used to control how requests flow between services.
- Promote traffic resilience with features like circuit breaking, retries, and timeouts.
- Simulate failures to test the resilience of your services and identify potential weaknesses with fault injection.

Google Cloud

CSM allows for several advanced traffic management features.

Traffic routing can be used to control how requests flow between services, enabling canary deployments, A/B testing, and blue/green deployments.

Promote **traffic resilience** with features like circuit breaking, retries, and timeouts. Make your applications more robust and prevent cascading failures.

Simulate failures to test the resilience of your services and identify potential weaknesses with **fault injection**.

Proprietary + Confidential

Observability

- Observability in GKE is critical and can be complex.
- To simplify the observability management of GKE clusters, Google offers several features including monitoring, tracing, and logging.

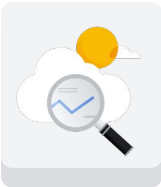
Google Cloud

Another advantage is observability and visibility into your clusters and infrastructure. Observability in GKE is critical and can be complex.

To simplify the observability management of GKE clusters, Google offers several features including monitoring, tracing, and logging.

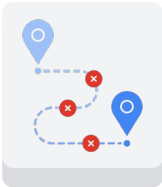
Proprietary + Confidential

Observability tools



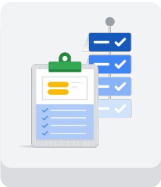
Cloud Monitoring

Gain insights into service performance with detailed metrics like request latency, error rates, and throughput.



Cloud Trace

Follow requests as they travel through your application to pinpoint bottlenecks and debug issues.



Cloud Logging

Capture detailed logs of service-to-service communication for auditing and troubleshooting.

Google Cloud

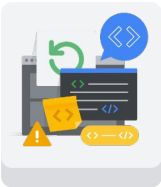
Cloud Monitoring allows administrators to gain insights into service performance with detailed metrics like request latency, error rates, and throughput.

Cloud Trace will follow requests as they travel through your application to pinpoint bottlenecks and debug issues.

Capture detailed logs of service-to-service communication for auditing and troubleshooting with **Cloud Logging**.

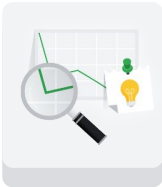
Proprietary + Confidential

Observability tools benefit multiple teams



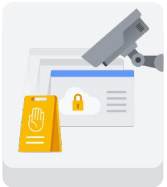
Developers

Write cleaner code by focusing on business logic, and worry less about networking complexities.



Operations

Gain greater control over traffic management, security, and observability, making it easier to manage and troubleshoot applications.



Security

Enforce security policies consistently across all services, reducing the risk of vulnerabilities.

Google Cloud

These tools positively impact multiple teams in an organization.

Developers can write cleaner code by focusing on business logic, and worry less about networking complexities.

Operations Teams gain greater control over traffic management, security, and observability, making it easier to manage and troubleshoot applications.

And **Security Teams** can enforce security policies consistently across all services, reducing the risk of vulnerabilities.

Proprietary + Confidential

Cloud Service Mesh benefits

- Faster development cycles
- More reliable applications
- Increased business agility

Google Cloud

Ultimately, Cloud Service Mesh can lead to faster development cycles, more reliable applications, and increased business agility.

Proprietary + Confidential

Cloud Service Mesh

01 Introduction to Cloud Service Mesh

02 Provisioning Cloud Service Mesh

03 Handling requests

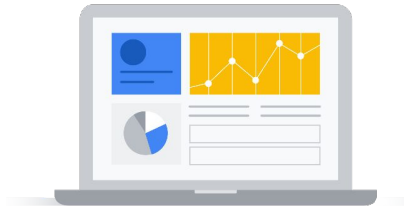
04 Cloud Service Mesh dashboards and support

Google Cloud

Before you can leverage the robust features of Cloud Service Mesh, you must connect the cluster to a fleet, and provision CSM. This section describes how to provision Cloud Service Mesh.

Proprietary + Confidential

Enable the Cloud Service Mesh API



Google Cloud console

```
gcloud services enable \
mesh.googleapis.com \
--project=FLEET_PROJECT_ID
```

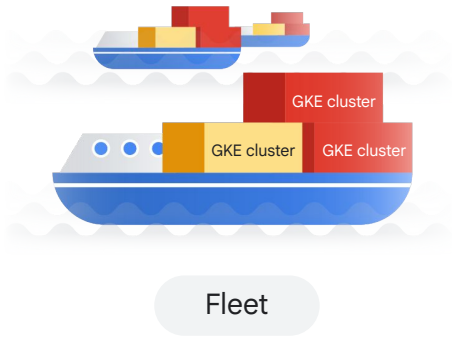
gcloud command

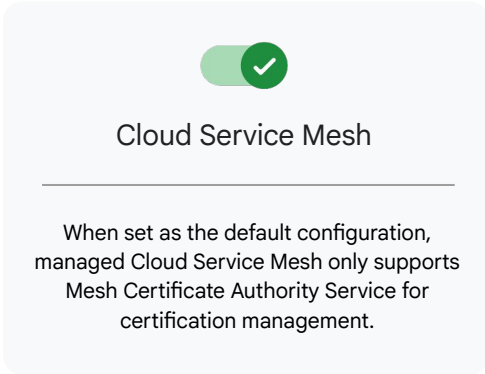
Google Cloud

The first step is to enable the **Cloud Service Mesh API** in the Google Cloud console, or by using the `gcloud services enable` command.

Proprietary + Confidential

Managed Cloud Service Mesh as a default configuration





Google Cloud

You can choose to use managed Cloud Service Mesh as a default configuration for your fleet.

With this configuration, CSM will be automatically enabled for each newly registered GKE cluster.

When set as the default configuration, managed Cloud Service Mesh only supports Mesh Certificate Authority Service for certification management.

Proprietary + Confidential

Set up automatic management for CSM

mesh.yaml

```
management: automatic
```

Provision a control plane for CSM

```
gcloud container fleet \
  mesh enable --project
   FLEET_PROJECT_ID \
--fleet-default-member-confi
        g mesh.yaml
```

Add the existing cluster

```
gcloud container \
clusters update \
CLUSTER_NAME --location \
CLUSTER_LOCATION \
--fleet-project \
FLEET_PROJECT_ID
```

Verify the control plane

```
gcloud container fleet \
mesh describe
```

Google Cloud

To set up automatic management for CSM across all the Kubernetes clusters in your Google Cloud fleet, you enable CSM for your fleet.

The first step is to create a **mesh.yaml** file with the `management: automatic`

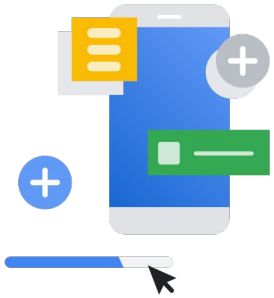
Now use the `gcloud container fleet mesh enable` command to provision a control plane for CSM.

Verify by using the `gcloud container fleet mesh describe` command.

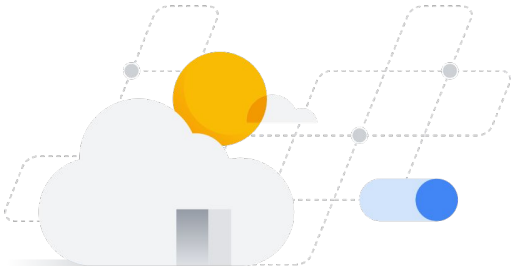
An **ACTIVE** state indicates the process is complete.

Proprietary + Confidential

Enable applications to use the mesh



Application



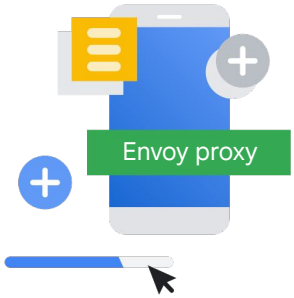
Cloud Service Mesh

Google Cloud

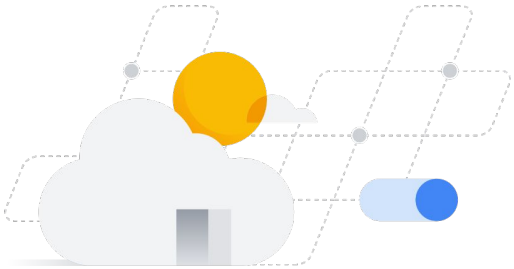
Now the mesh has been created, but applications are not using it yet. For applications to use the mesh,

Proprietary + Confidential

Enable applications to use the mesh



Application



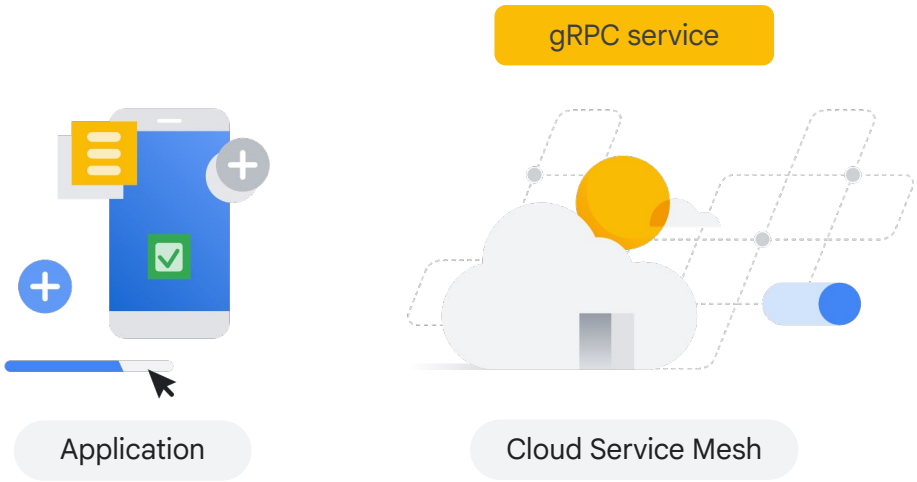
Cloud Service Mesh

Google Cloud

you either need to inject Envoy proxies

Proprietary + Confidential

Enable applications to use the mesh



Google Cloud

or establish a proxyless gRPC service within your service mesh.

Proprietary + Confidential

Automatically inject sidecars at the namespace level

Add a label to the namespace

```
kubectl label namespace \
sidecar-example \
istio-injection=enabled \
istio.io/rev-
```

Automatically update sidecar proxies

```
kubectl annotate \
--overwrite namespace \
sidecar-example \
mesh.cloud.google.com/pro
xy='{"managed":"true"}'
```

Verify the injection

```
kubectl get pods -n \
sidecar-example -l \
istio-injection=enabled
```

Google Cloud

For production scale applications, you need an automatic way to inject sidecars at the namespace level. This allows your administrators to organize resources. It is also useful in defining security and connectivity boundaries.

You can use the `kubectl label namespace sidecar-example` command to add a label to the namespace.

Any client pods created in that namespace will have an Envoy sidecar container automatically injected.

Automatically injecting sidecars is more efficient than injecting them manually.

You can also enable Google to manage your data plane so that the sidecar proxies will be automatically updated for you.

This is done by annotating the namespace with the kubectl annotate command.

You can verify that the sidecar container was injected by running the kubectl get pods command. Specify the namespace and label you'd like to verify.

Proprietary + Confidential

Verify that the service mesh is set up

```
$ kubectl get pods,svc -n sidecar-example
NAME                                READY    STATUS RESTARTS    AGE
pod/client-6c9dd4869b-rbvnn         2/2   Running  0           2h
pod/whereami-7bd48c756c-xntxg       2/2   Running  0           2h
pod/whereami-7bd48c756c-xp46z       2/2   Running  0           2h

NAME               TYPE      CLUSTER-IP   EXTERNAL-IP  PORT(S)   AGE
service/whereami   CluserIP 34.118.232.36  <none>       8080/TCP  2h

$ CLIENT_POD = $(kubectl get pod -n sidecar-example -l run=client
-o=jsonpath='{.items[0].metadata.name}')
TEST_CMD = "curl whereami.sidecar-example.svc.cluster.local:8080"

kubectl exec -it $CLIENT_POD -n sidecar-example -c client -- /bin/sh -c "$TEST_CMD"
```

Google Cloud

To verify that the Envoy-sidecar service mesh is set up, you can send a request to the **whereami** service from the client.

Proprietary + Confidential

Verify that the service mesh is set up

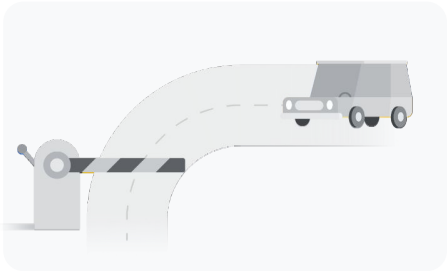
```
*  Host whereami.sidecar-example.svc.cluster.local:8080 was resolved
*  IPv6: (none)
*  IPv4: 34.118.232.36
*   trying 34.118.232.36:8080...
*  Connected to whereami.sidecar-example.svc.cluster.local (34.118.232.36) port 8080
*  using HTTP/1.x
>  GET / HTTP/1.1
>  Host: whereami.sidecar-example.svc.cluster.local:8080
>  User-Agent: curl/8.13.0
>  Accept */*
>
*  Request completely sent off
<  HTTP/1.1 200 OK
<  server: envoy
<  date: Mon, 19 May 2025 06:38:09 GMT
<  content-type: application.json
<  content-length: 368
```

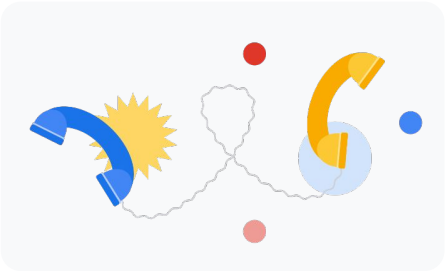
Google Cloud

In this example, all requests from the client container are redirected to the Envoy proxy sidecar, so you can use any IP address.

Proprietary + Confidential

Proxyless gRPC services





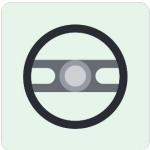
Proxyless gRPC applications

Google Cloud

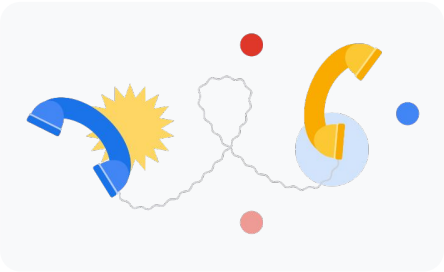
An alternative to a proxy sidecar is to use a proxyless gRPC service to allow applications to start using a mesh.

Proprietary + Confidential

gRPC configuration options



Automatic



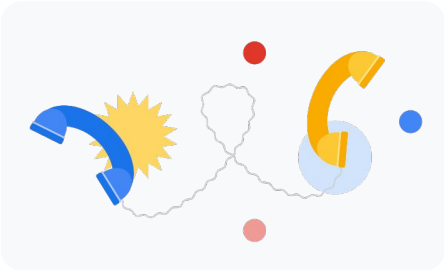
Manual

Google Cloud

Automatic gRPC configuration simplifies the setup process, but if your infrastructure requires more granular control, you can configure it manually. For details on how to accomplish this, please refer to the Google Cloud documentation.

Proprietary + Confidential

The proxyless gRPC service



- To use proxyless gRPC service, your application needs to be working with gRPC already.
- Often reserved for high-performance gRPC applications and resource efficiency in a large-scale service meshes.
- Where you can't deploy sidecar proxies, using regular CSM.

Google Cloud

To use proxyless gRPC service, your application needs to be working with gRPC already.

- This approach is often reserved for specific use cases:
- High-performance gRPC applications
- Resource efficiency in a large-scale service meshes

Where you can't deploy sidecar proxies with regular CSM.

Proprietary + Confidential

Sidecar proxies intercept traffic, add mTLS, and collect telemetry data

Google Cloud

After the sidecar proxies are up and running, they will intercept traffic, add mTLS when possible, and collect telemetry data. This data is accessible from the Cloud Service Mesh console.

Proprietary + Confidential

How do sidecars configure the pod's networking to intercept requests?

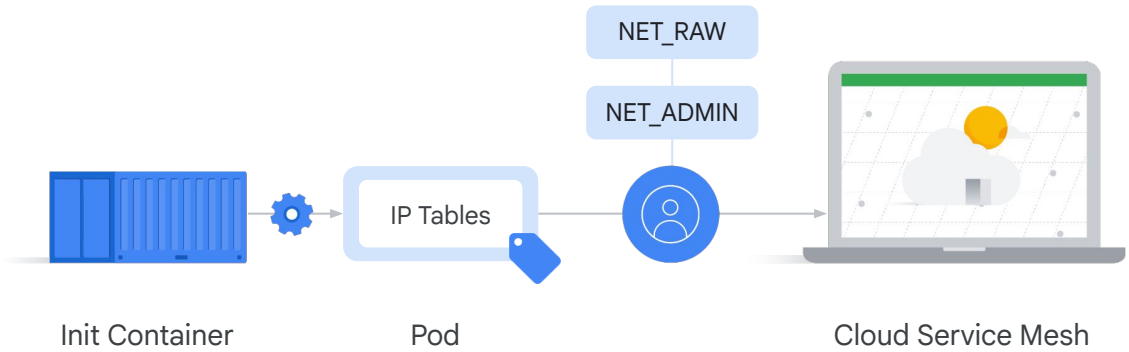
Google Cloud

So, how do the sidecars configure the pod's networking to intercept all requests?

There are two options depending on your mesh configuration.

Proprietary + Confidential

Init containers configure the IP tables by default



Google Cloud

An Init Container configures the IP Tables of the pod by default. The user or service-account deploying pods to the mesh must have Kubernetes RBAC permissions to deploy pods with the NET_ADMIN and NET_RAW.

Granting elevated Kubernetes RBAC permissions can lead to security compliance issues for some organizations.

Proprietary + Confidential

The Istio CNI plugin configures traffic redirection

Google Cloud

The second option is to enable the Istio CNI plugin. This plugin configures traffic redirection during the network setup phase of a Kubernetes pod's lifecycle.

This option does not require elevated Kubernetes RBAC permissions.

Proprietary + Confidential

Cloud Service Mesh

01 Introduction to Cloud Service Mesh

02 Provisioning Cloud Service Mesh

03 Handling requests

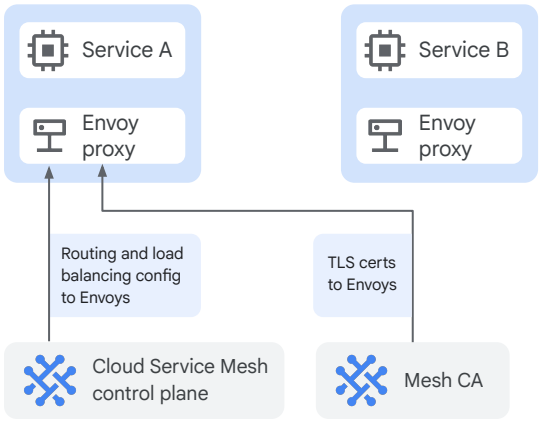
04 Cloud Service Mesh dashboards and support

Google Cloud

Now that we've covered the basics of Cloud Service Mesh, let's dive into the request handling process.

Proprietary + Confidential

Envoy proxies gathers information from the control plane



Google Cloud

When a pod is created, your workload container, in this case named “Service A,” starts up. For this example, Service A is on the client side and Service B is on the server-side.

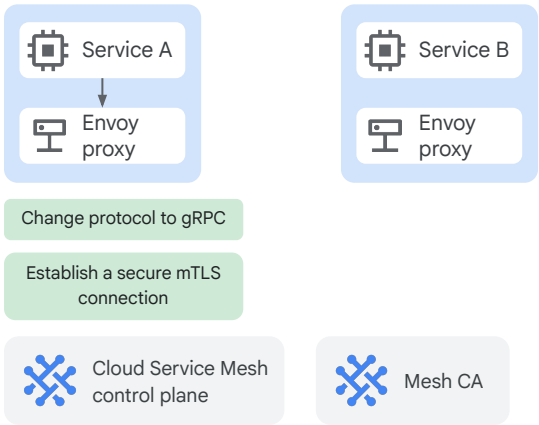
Then the Envoy sidecar proxy container is injected into the pod, and IP tables are configured so that all requests are redirected to Envoy.

Envoy gathers service information, routing, and configuration policy from the control plane so that it knows where to send requests.

Finally, the **Service Mesh certificate authority**, in this case Mesh CA, securely distributes TLS certificates to the Envoy proxies, enabling them to establish secure, mutually authenticated connections (mTLS).

Proprietary + Confidential

Envoy determines how and where to route the call



Google Cloud

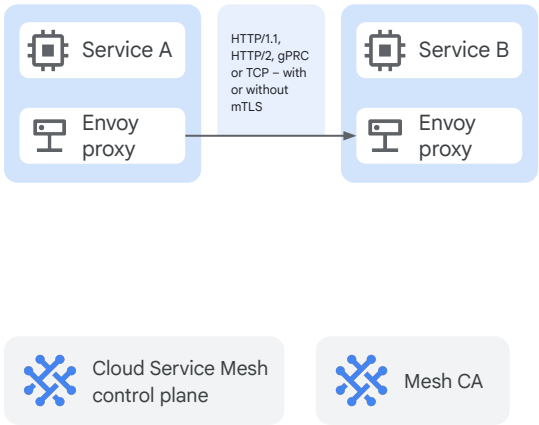
When Service A places a call to service B, the client-side Envoy proxy intercepts the call.

Envoy consults the local configuration to determine how and where to route the call to service B.

For example, Envoy might change the protocol to gRPC and establish a secure mTLS connection.

Proprietary + Confidential

Envoy forwards the request to service B



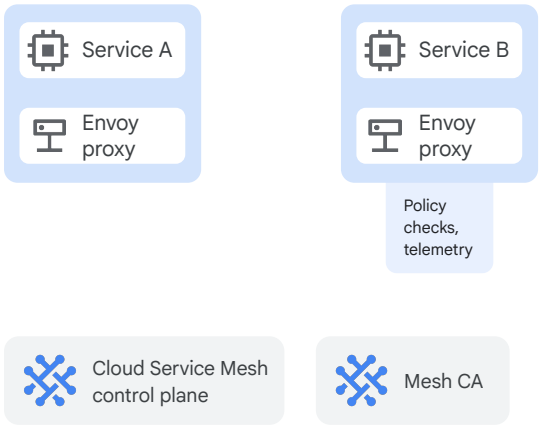
Google Cloud

Then Envoy forwards the request to the appropriate instance of service B.

The Envoy proxy deployed with the service intercepts the call, validates certificates, and establishes the mTLS connection.

Proprietary + Confidential

Envoy filters are used to validate calls

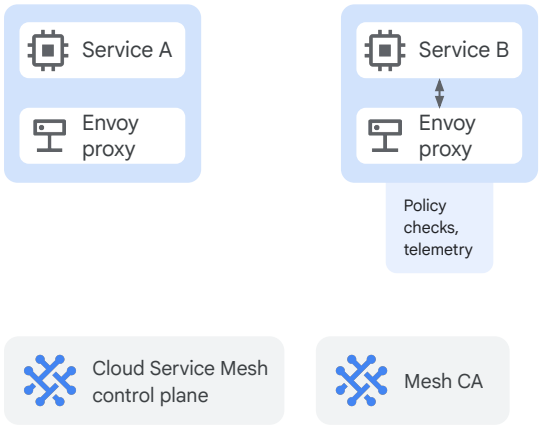


Google Cloud

For validation, the receiving Envoy uses Envoy filters. These filters extend the functionality of Envoy proxies. For example, they can be used to enforce access control policies, perform quota checks, and collect telemetry data.

Proprietary + Confidential

Envoy forwards the request to Service B

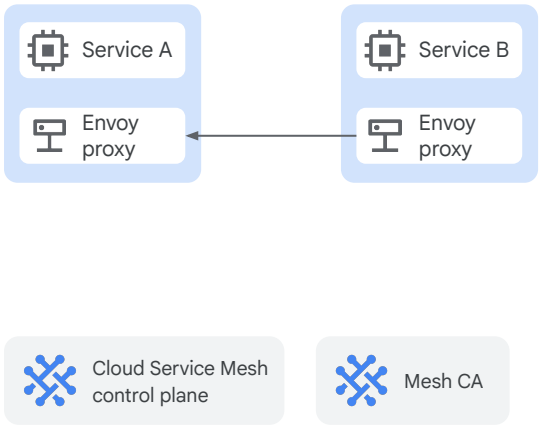


Google Cloud

After the call is validated, the server-side (destination) Envoy forwards the request to service B.

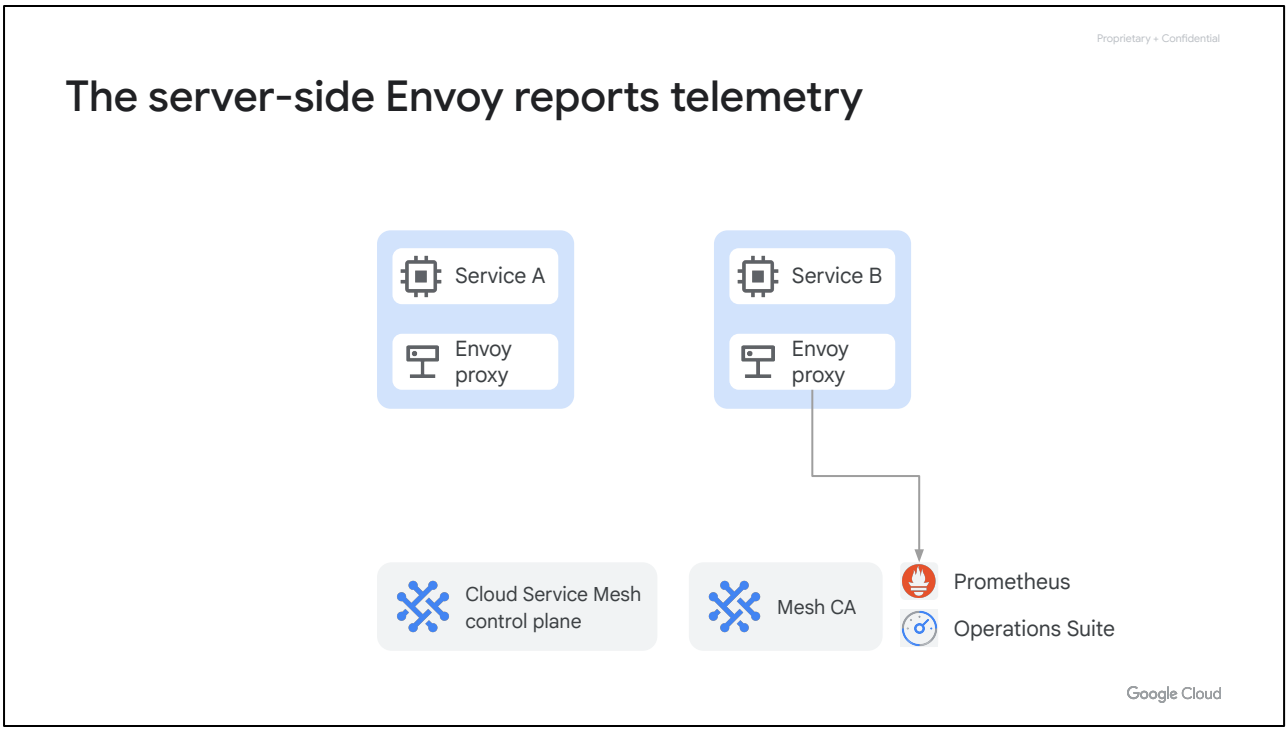
Proprietary + Confidential

The response is intercepted by the client-side caller



Google Cloud

Service B then processes the request and sends a response back, which is intercepted by the Envoy proxy of Service A.



At this stage, the server-side Envoy is done with the request and reports telemetry to in-proxy extensions called WebAssembly (Wasm) extensions. These extensions let developers add custom networking functionality by notifying the appropriate plugins.

However, when considering Cloud Service Mesh, the operational philosophy shifts. CSM operates as a fully managed service and Google Cloud takes on the responsibility of maintaining the Envoy proxies.

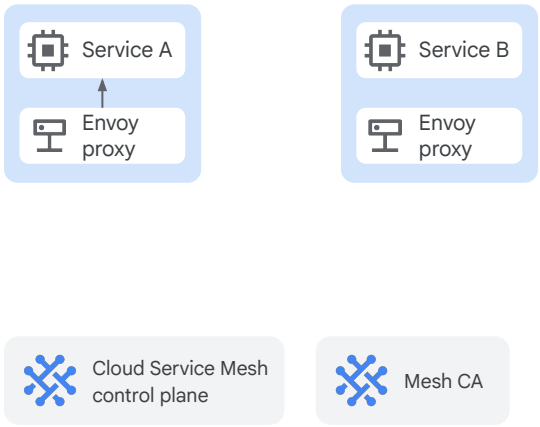
This restriction exists because injecting unvetted Wasm modules directly into the managed Envoy proxies introduces potential vulnerabilities, and stability risks, that are incompatible with the managed service model.

Therefore, while Envoy itself is capable of running Wasm extensions, the nature of CSM's managed service limits the direct user implementation of the technology, in favor of a more stable and secure service.

-

Proprietary + Confidential

The client-side Envoy forwards the response to the original caller



Google Cloud

Finally, the client-side Envoy proxy forwards the response to the original caller, Service A.

Proprietary + Confidential

Cloud Service Mesh

01 Introduction to Cloud Service Mesh

02 Provisioning Cloud Service Mesh

03 Handling requests

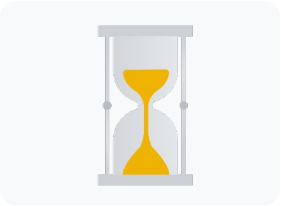
04 Cloud Service Mesh dashboards and support

Google Cloud

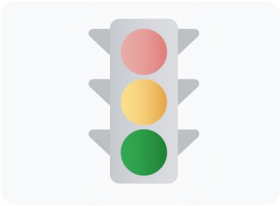
Service dashboards in Cloud Service Mesh provide critical, service level metrics for all services in your mesh.

Proprietary + Confidential

Golden signals

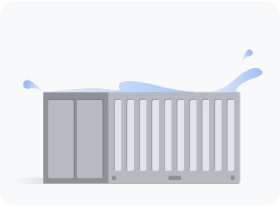


Latency



Traffic

Error



Saturation

Google Cloud

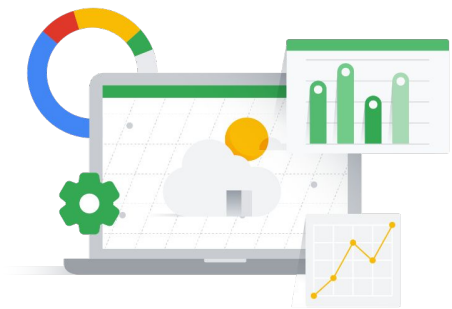
These dashboards provide insight into three of the four golden signals of monitoring: latency, traffic, error, and saturation.

This course does not go into detail about saturation.

While a dedicated saturation metric might not exist, the CSM dashboard provides the essential metrics (CPU, memory, latency, traffic) needed to determine saturation.

Proprietary + Confidential

CSM dashboards



- Define, review, and set alerts against service level objectives (SLOs).
- View detailed charts for individual services, filter the data, and drill into elements like response code, protocol, destination pod, and traffic source.
- Observe traffic flow between services and communication performance.

Google Cloud

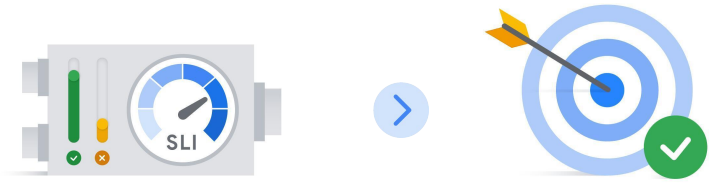
With these dashboards, you can define, review, and set alerts against service level objectives, or SLOs, which summarize your service's user-visible performance.

You can view detailed charts for individual services, filter the data, and drill into elements like response code, protocol, destination pod, and traffic source.

Service endpoint information is also available, enabling you to observe traffic flow between services and communication performance.

Proprietary + Confidential

SLIs and SLOs



Service Level Indicators

Service Level Objectives

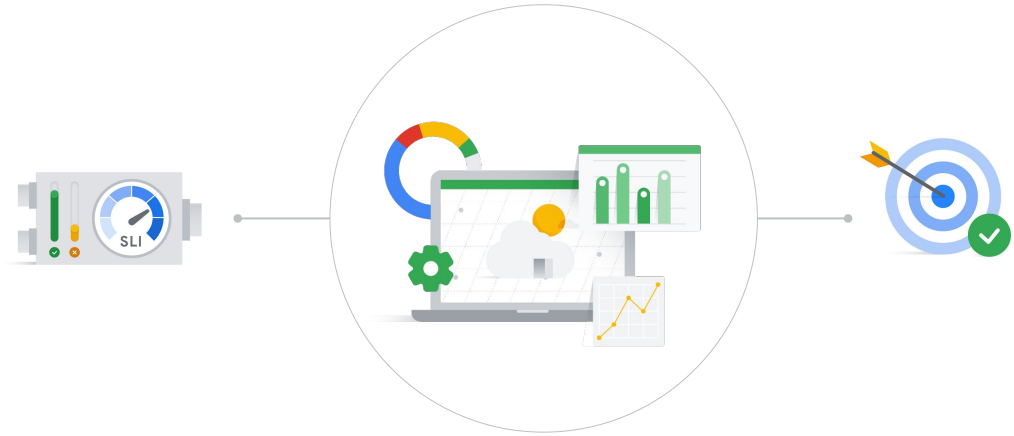
Google Cloud

Service level indicators (SLIs) are the key metrics you must report on, for example, latency and availability.

After figuring out what to measure (SLIs), you can set targets for how good they need to be (SLOs). This helps you determine how much room for error you have (error budget).

Proprietary + Confidential

Centralized monitoring with CSM

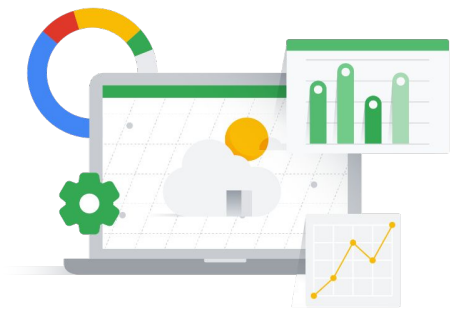


Google Cloud

CSM provides a central, unified location for service operators to monitor and evaluate the health of their services. Alerts can be created to notify operators when an SLO has no more room for error.

Proprietary + Confidential

Mesh topology, traffic flow, workloads, and query metrics



- Visualize mesh topology.
- View relationships and traffic flow between services.
- Drill into workloads and queries per second.

Google Cloud

You can also visualize your mesh topology in CSM dashboards.

Service charts are automatically generated, and display the relationships and traffic flow between services.

Finally, you can drill down into the workloads and pods of a service, and queries per second (QPS) rates between services.

Proprietary + Confidential

CSM is available with GKE or as a standalone offering

Billing is determined by the Google Cloud APIs enabled on your project.

Google Cloud

CSM is available with GKE or as a standalone offering on Google Cloud.

Billing is determined by the Google APIs that are enabled on your project.

Proprietary + Confidential

Using Cloud Service Mesh as a standalone service

- Compute Engine VMs and GKE pods.
- Telemetry dashboards with standard metrics. Custom metrics are charged based on Cloud Monitoring pricing.
- A CSM-managed control plane.
- Mesh CA, a managed certificate authority service, with no per-certificate charge.

Google Cloud

When using Cloud Service Mesh as a standalone service, pricing is based on the number of clusters and the number of Service Mesh clients.

Cloud Service Mesh as a standalone service includes:

- Compute Engine VMs and GKE pods.
- Telemetry dashboards with standard metrics. Custom metrics are charged based on Cloud Monitoring pricing.
- A CSM-managed control plane.
- Mesh CA, a managed certificate authority service, with no per-certificate charge.

Proprietary + Confidential

Cloud Service Mesh Limitations compared to Istio

- Service mesh integration with custom CAs.
- Usage of Envoy filters to extend the service mesh for additional telemetry and policy.
- Usage of arbitrary telemetry and logging backends.
- Multi-network support and IPv6 support for Kubernetes is also only available in Istio.
- Ambient mode data plane mode.

Google Cloud

As of version 1.23, Cloud Service Mesh has some limitations compared to Istio.

Limitations include:

- Service mesh integration with custom CAs. With Istio, you can set up your own custom root CA which might manage identities both inside and outside of your cluster.
- Usage of Envoy filters to extend the service mesh for additional telemetry and policy. With Istio, you can extend the default functionality to provide additional checks at the networking layer.
- Usage of arbitrary telemetry and logging backends. Service Mesh provides an out-of-the-box integration with Google Observability, but it’s not possible to configure additional backends.While you can utilize the Prometheus metrics for your own monitoring solutions, the direct configuration of alternative logging or telemetry systems beyond Google's is not supported.
- Multi-network support and IPv6 support for Kubernetes is also only available in Istio.
- Ambient mode data plane mode. With Istio, you can configure a per-node Layer 4 proxy, and optionally a per-namespace Envoy proxy for Layer 7 features instead of having a sidecar proxy on a per pod basis.